Role Description Cards — Project 2

Five-Person Team, Weeks 5–9 (PDE BVPs)

Working roles for coordination, not walls: every team member should be able to run the code and read the report.

These are **suggested** role descriptions, not a rigid requirement — they exist to help a five-person team self-organise. You may rename, merge, or redistribute the responsibilities; what matters is that (i) every deliverable has an owner, and (ii) nobody waits for an upstream "expert" before starting. **Roles rotate from Project 1**: if you were the Algorithm Implementation seat last block, take a different seat now.

Each card says what the role typically owns, what "done" looks like, and how it is checked. Put them on the module homepage at the start of Week 5.

---

1. Project Manager

Owns: the plan, the meetings, the presentation, the git setup, and the weekly records (challenge consensus + milestone).

- Runs a 15-minute team meeting before each Tutorial (arrive a little early) or at the start of the tutorial's team-work block — the opening micro-lecture slot is not available for it
- Maintains a simple Gantt table (what / who / due) on the module homepage
- Makes sure every teammate has a named task for the week
- Creates the **Project 2 repository** in Week 5 (roles rotated), so all collaboration happens through git from day one
- Invites every teammate *and the teacher* as repository collaborators (Write access) in Week 5, so everyone — teacher included — can clone and push from day one
- Coordinates the seminar presentation (voluntary first wave in Week 8, or second wave in Week 9 — the team presents once)
- Records the team's **challenge consensus** at each Tutorial's review — the one result everyone agrees on
- Posts the team's **Weekly-Milestone** row at each wrap-up (what we completed / what we plan next / current blocker)
- Posts the **team result on the course forum** after each Tutorial's review (one post per team per week — the team record)
- Coordinates the final submission (demo + repo + report deadline)

"Done" looks like: the Gantt table is updated each Tutorial; the repository exists with `code/`, `figures/`, `report/` layout and every teammate has pushed by the end of Week 5; the slides exist before the assigned seminar; the Weekly-Milestone row and the team's forum post appear every week; the final submission is in on time; nobody is idle in a tutorial.
Checked by: the team's checkpoints each week + presentation quality + the weekly milestone and forum posts + the challenge-consensus rows.

2. Mathematical Theory

Owns: every formula that appears in the report.

- States the continuous problem: PDE, domain, boundary decomposition, coefficients, and the compatibility condition if the problem is pure-Neumann
- Derives the truncation error of the five-point stencil and the boundary closure (one-sided vs. ghost point) with its order
- Writes down the weak form and the trial/test spaces if FEM is used
- Owns the regularity discussion (what rate does the theory actually promise on your domain?)
- Proofreads all equations in the final LaTeX

"Done" looks like: the report's model and verification sections contain derivations the whole team can defend — including the boundary row, coefficient by coefficient.
Checked by: seminar Q&A — be ready to derive on the whiteboard the closure whose order you claim.

3. Algorithm Implementation

Owns: the code that solves the problem.

- Completes `poisson_fd_skeleton.py` in Week 5; keeps f, the coefficient, and the boundary data swappable
- Builds the sparse system in COO, solves in CSR; keeps the $(i, j) \mapsto k$ index map explicit
- Adds (if chosen) the iterative solvers, the 1D FEM scaffold, or a non-rectangular domain mask
- Commits regularly with meaningful messages (end of each session)
- Keeps `run_all.py` running end-to-end to reproduce every figure

"Done" looks like: `python code/run_all.py` regenerates every figure in the report on a machine with the dependencies installed.
Checked by: code-review checklist (Week 8) + the viva questions.

4. Visualization & Report

Owns: every figure and the written report.

- Produces the solution surface, the `spy` plot of the matrix, the refinement log–log plot with a reference slope, and the cost-versus-accuracy comparison
- Uses `convergence_plot_template.py` so every refinement figure states the fitted order and the fit deviation
- Typesets the LaTeX report and the slides
- Keeps the English clean and the narrative clear

"Done" looks like: the report reads like a detective story (problem → method → evidence → interpretation), and every figure has a caption explaining what is shown and why it matters.
Checked by: the visualization guide + reviewer comments.

5. Testing & Validation

Owns: whether the results can be *believed*.

- Runs the manufactured-solution bank (`test_problem_bank.py`) against the team's solver in Week 5, before any "interesting" problem is attempted
- Owns the four-mesh refinement protocol, the named norm, and the observed-order fit — including explaining deviations
- Separates algebraic error from discretisation error (residual vs. tolerance sweep for iterative solvers, backward error and conditioning for a direct solve) before the clinic
- Hunts edge cases: boundary sign errors, index-map transposition, tolerance contamination, degenerate coefficients

"Done" looks like: every key number in the report is backed by a manufactured-solution check or a refinement table, and the clinic readiness gate passes in Week 7.
Checked by: the instructor runs the code; the numbers should match the team's own verification evidence.

Rotation note

Roles rotate between projects: by the end of the course every student should have tried at least three different seats. If someone was clearly the strongest coder in Project 1, it is still worth letting someone else take the Algorithm Implementation seat here — but follow your team's judgement.

What all five share

- You can explain the five-point stencil and the boundary treatment even if the code is not "your" part
- You attend every session and submit your issue ticket by Tuesday
- You write your Individual Contribution Statement (ICS) for the final report
- You are honest about what you did and did not do

Example work plan — parallel, not serial

All five roles start in Week 5; deliverables mature at different weeks. The columns show what each person should have moving in a given week — they are *not* a hand-off sequence.

	Week 5	Week 6	Week 7	Week 8	Week 9
PM	new repo, roles + Gantt	checkpoints	clinic pitch logistics	rehearse (first wave?)	submission manifest
Math	PDE + BC statement, closure derivation	weak form (if FEM), solver orders	regularity vs. observed rate	proofread, limitations	final proofread
Algo	stencil + assembly on the square	boundary treatment, iterative solvers	tolerance sweep, extensions	`run_all.py` end-to-end	freeze code
Valid	MMS bank green	first refinement table	four-mesh protocol, clinic gate	reproduction checks	residual audit
Viz	report skeleton, spy + surface plots	log–log with reference slope	clinic figure polish	main figures + slides	final report

Note the BVP-specific timing: verification starts in Week 5 (the manufactured bank) and plots start in Week 6 (the first refinement table) — nobody waits for a finished upstream. The Week 7 clinic is formative: it exists to catch incoherent discretisation/norm/tolerance triples before they reach the report.